# Project 2 - Minesweeper-o-Matic

CSU44012
Leah Weldon
21363670

# 1. Introduction

This report details my Minesweeper-o-Matic project, which implements the classic Minesweeper puzzle in Haskell. The assignment brief required two main phases: first, constructing a Haskell model of Minesweeper in Phase 1, and second, providing a user interface with Threepenny GUI and an automatic solver in Phase 2. My solution supports fully interactive play, with random mine placement, an effective "auto-play" button for logically safe moves, and user-friendly feedback through a minimal yet cohesive GUI.

# 2. Phase 1 Implementation

## 2.1 Data Model and Randomisation

The Minesweeper logic is primarily encapsulated in my Mine.hs. A Board data type maintains a 2D list of BoardCells, each represented as Cell CellKind Display. The CellKind can be either Bomb or Open Int (indicating how many bombs are adjacent), while Display captures whether a cell is Covered, Uncovered, or Flagged.

1. **Random Bomb Placement**
   To fulfill the requirement of distributing mines randomly, I used the Fisher-Yates shuffle from my Lib.hs. This shuffle-based approach shuffles all possible board coordinates and takes the first n positions as bomb locations (where n is the user-specified number of mines, another feature I added). The function fillAdjCounts then computes each safe cell's adjacency count.
2. **Strongly Typed Cells**
   The typed approach ensures minimal risk of "accidental" states—like uncovering a flagged cell—because each function precisely matches patterns (Covered, Flagged, Uncovered). This clarity not only helped prevent errors but also made the debugging process more straightforward.

## 2.2 Interactive Gameplay

After building the data model, I allowed users to uncover and flag cells:

- **Reveal a Cell**
  If the user is in "Expose Mode," left-clicking an unexposed cell calls revealCell. If the cell is a bomb, the program triggers an immediate GameOver. If safe, and the adjacency count is zero, it expands outward in a ripple effect, unveiling neighboring zeroes.
- **Flag a Cell**
  In "Flag Mode," clicking toggles a covered cell to flagged or vice versa. This distinction protects the user from accidentally revealing suspected bombs.

All these interactions return a new Board state and updated outcome where relevant. The purely functional nature of Haskell's approach made these transformations deterministic and transparent.
I tested these interactions by generating multiple boards of various sizes, ensuring that a flagged cell remained inaccessible to accidental reveals. Whenever I encountered unexpected behaviour like maybe a ripple reveal not terminating properly, I used print statements in my Haskell code to trace adjacency logic. This step-by-step approach helped me confirm the correctness of both single-cell exposes and multi-cell expansions.

### 2.3 Endgame Condition

checkVictory verifies if every safe cell is uncovered. If the user hits a bomb, revealAllBombs opens all bombs to show exactly where they were. Therefore, the program can easily transition to a final state. Another addition I added during phase 1 was a Landing page. Screenshot 1 shows the Landing Page where instructions, and the "Ready? Let's Play!" button are displayed. This page is hidden once the user proceeds to the game.

## 3. Phase 2 Implementation

### 3.1 Threepenny GUI and Layout

My front-end resides in GameUI.hs. Upon choosing the board dimensions with sliders (Rows, Columns, Mines), pressing "New Board" calls initBoard to generate a random arrangement. Each cell is drawn as an SVG rectangle: covered cells appear pinkish, while uncovered cells show white backgrounds with bombs, flags, or adjacency counts.

Event wiring uses IORefs to store and modify the board. Left-click or right-click events are detected through Threepenny. I also integrated a timer to record how many seconds pass until the user either wins or loses as another cool feature.

Screenshot 2 illustrates the Board Page, highlighting how flagged cells, uncovered numbers, and bombs appear during gameplay. Once a bomb is clicked, the game ends, and the timer stops.

### 3.2 Auto-Play: Simple and Advanced Moves

A button labeled "Play Move" invokes autoMove from Mine.hs, which tries:

1. **Advanced Patterns**
   If a revealed cell has adjacency number 1 or 2, and a precise arrangement of flagged neighbors matches that number, the remaining neighbors are guaranteed safe or bombs.
2. **Basic Patterns**
   For instance, if a cell's number equals the count of flagged neighbors, the rest of its covered neighbors must be safe, or if the number equals the total covered neighbors, they must be bombs.
3. **Fallback Guess**
   If no definite pattern emerges, the solver uncovers a random covered cell. This ensures at least one move is taken, even though it involves risk.

While these rules will not solve every puzzle, they do satisfy the assignment's requirement to "play a move automatically if there is an unambiguously safe move available."

## 4. Reflection on Development and Language

I discovered Haskell to be a strong fit for the logic-driven aspects of Minesweeper. The pattern-matching syntax simplified adjacency checks and expansions for zero-valued cells, while the static type system caught potential inconsistencies early. Testing usually involved generating multiple boards to confirm correct bomb counts and verifying that revealing zero cells expanded properly. Debugging was primarily straightforward, though Threepenny occasionally required small JavaScript snippets (for instance, to capture right-click events or to redraw the board container). Yet, that mild inconvenience was offset by

the clarity of Haskell's event-based approach, which neatly segregates pure game logic from side-effectful UI code.

Regarding software development under time constraints, I found it best to isolate smaller test scenarios: focusing on zero expansions first, then verifying bomb reveals, and finally layering on the auto-play solver. The advanced patterns occasionally took extra reasoning, but pattern matching to a known adjacency (like 1 or 2 around flagged neighbours) was direct once I enumerated the relevant cases. In future expansions, probability-based moves or a more comprehensive solver is something I'd like to look at attempting.

# 5. Conclusion

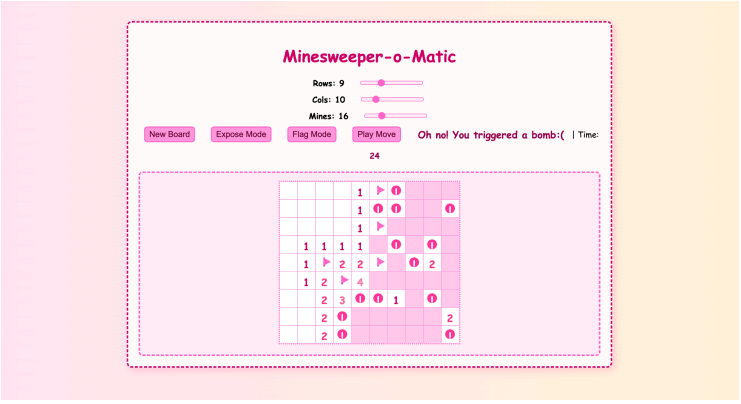
## 5.1 Phase 1 Outcomes

- **Random Bomb Distribution**: The function initBoard uses the Fisher-Yates shuffle from Lib.hs to uniformly distribute bombs.
- **Interactive Reveal/Flag**: Users can safely mark bombs in "Flag Mode" or uncover cells in "Expose Mode."
- **End Condition**: The game halts if a bomb is uncovered (GameOver) or if all safe cells are revealed.

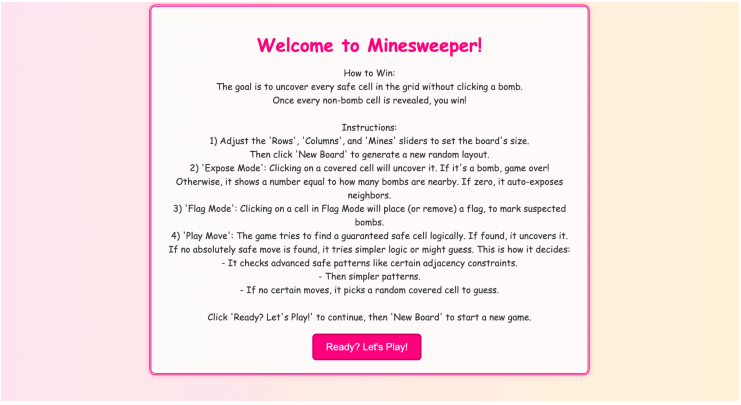
## 5.2 Phase 2 Outcomes

- **GUI with Sliders**: Through Threepenny, players define board size and mine count, press "New Board," and begin.
- **Auto-Player**: "Play Move" calls autoMove, which attempts advanced or basic logic before guessing. This meets the project's solver requirement for unambiguously safe moves.

In summary, my Minesweeper-o-Matic project addresses both phases of the assignment brief. I leveraged Haskell for reliable bomb distribution, adjacency checks, and expansions, while a minimal Threepenny interface provides real-time user interaction, a timer, and solver assistance. The end result is a game that demonstrates functional programming's clarity, practical performance, and expressive pattern-matching advantages.




*Screenshot 1*




*Screenshot 2*